

LAPORAN PRATIKUM
STRUKTUR DATA
“Laporan Praktikum Pekan 7”

Disusun oleh:

Rifal maulana oskar

2511533024

Dosen Pengampu : Dr. Wahyudi, S.T, M.T

Asisten Praktikum : Sherly Sukmadira



DEPARTEMEN INFORMATIKA
FAKULTAS TEKNOLOGI INFORMASI
UNIVERSITAS ANDALAS

2025

KATA PENGANTAR

Puji syukur ke hadirat Allah SWT karena berkat rahmat dan karunia-Nya, penulis dapat menyelesaikan laporan praktikum ini dengan baik.

Laporan ini disusun untuk memenuhi tugas praktikum Struktur Data pada pertemuan ke enam dengan topik dasar pemrograman Java, yaitu pembuatan program Sorting Algorithm.

Semoga laporan ini dapat memberikan pemahaman dasar mengenai penggunaan bahasa pemrograman Java, khususnya dalam mengenal struktur dasar program, fungsi, serta pemanggilan metode.

Penulis menyadari laporan ini masih jauh dari sempurna. Oleh karena itu, kritik dan saran yang membangun sangat diharapkan demi penyempurnaan laporan di masa mendatang.

Padang, 19 mei 2026

Rifal maulana oskar

DAFTAR ISI

BAB I	4
1.1 Latar belakang	4
1.2 Tujuan	5
1.3 Manfaat	5
BAB II	6
2.1 Langkah kerja program InsertionSort.	6
2.2 Contoh program InsertionSort.	6
2.3 Hasil output.	7
2.4 Analisis hasil dan Teori :	7
2.5 Langkah kerja program SelectionSort.	8
2.6 Contoh kode program SelectionSort.	8
2.7 Hasil output :	9
2.8 Analisis hasil dan teori :	9
2.9 Langkah kerja program BubleSort.	9
2.10 contoh kode program BubleSort.	10
2.12 Hasil output	11
2.13 Analisis hasil dan teori	11
2.14 Langkah kerja program InsertionSortGUI.	12
2.15 Contoh kerja program InsertionSortGUI.	12
2.16 Hasil output	18
2.17 Analisis hasil dan teori.	18
BAB III	19
DAFTAR PUSTAKA	20

BAB I

PENDAHULUAN

1.1 Latar belakang

Dalam dunia komputer dan pemrograman, pengolahan data merupakan salah satu aspek yang sangat penting. Data yang tersusun secara rapi dan teratur akan mempermudah proses pencarian, analisis, serta pengambilan keputusan. Oleh karena itu, dibutuhkan suatu metode yang dapat mengatur data agar tersusun secara terstruktur. Proses mengurutkan data ini dikenal dengan istilah *sorting* atau pengurutan.

Sorting merupakan teknik untuk menyusun data berdasarkan aturan tertentu, misalnya dari nilai terkecil ke terbesar (*ascending*) atau dari terbesar ke terkecil (*descending*). Algoritma sorting memiliki peran besar dalam bidang informatika, terutama dalam pengembangan aplikasi seperti sistem informasi, database, dan pengolahan data besar. Tanpa algoritma sorting yang baik, suatu program akan kesulitan mengelola data dalam jumlah banyak secara efisien.

Terdapat berbagai macam algoritma sorting yang dapat digunakan, namun beberapa algoritma yang paling dasar dan sering dipelajari dalam mata kuliah Struktur Data adalah **Insertion Sort**, **Selection Sort**, dan **Bubble Sort**. Ketiga algoritma ini termasuk dalam algoritma pengurutan sederhana, namun tetap sangat penting karena menjadi dasar untuk memahami algoritma sorting yang lebih kompleks seperti Merge Sort, Quick Sort, dan Heap Sort.

Insertion Sort bekerja dengan cara memasukkan elemen ke posisi yang sesuai pada bagian array yang sudah teratur. Selection Sort melakukan pengurutan dengan cara memilih elemen terkecil dari data yang belum teratur lalu menukarnya ke posisi depan. Sedangkan Bubble Sort melakukan pengurutan dengan cara membandingkan elemen bersebelahan dan menukarnya jika tidak sesuai urutan. Meskipun algoritma-algoritma ini tergolong sederhana dan memiliki efisiensi yang tidak terlalu tinggi pada data besar, namun mereka tetap banyak digunakan sebagai dasar pembelajaran konsep pengurutan.

Dengan mempelajari algoritma sorting seperti Insertion Sort, Selection Sort, dan Bubble Sort, mahasiswa dapat memahami bagaimana proses pengurutan data bekerja, serta dapat menganalisis kelebihan dan kekurangan dari masing-masing metode. Hal ini menjadi bekal penting dalam membangun program yang lebih optimal dan efisien.

1.2 Tujuan

Tujuan dari praktikum ini adalah:

- Untuk memahami langkah kerja dari algoritma Insertion Sort.
- Untuk memahami langkah kerja dari algoritma Selection Sort.
- Untuk memahami langkah kerja dari algoritma Bubble Sort.

1.3 Manfaat

Pembahasan ini diharapkan dapat membantu pembaca dalam:

- Memahami sintaks dan logika pada materi Sorting Algorithm.
- Mengimplementasikan kode syntax.

BAB II PEMBAHASAN

2.1 Langkah kerja program InsertionSort.

- Membuat kelas operator dalam package pekan7.
- Deklarasikan class utama dengan nama InsertionSort_2511533024.
- Masukan kode program.
- Run program.

2.2 Contoh program InsertionSort.

```
package pekan7_2511533024;

public class insertionsort_2511533024 {
    public static void inserttionsort_2511533024(int[] arr_3024) {
        int n_3024 = arr_3024.length;
        for (int i_3024 = 1; i_3024 < n_3024; i_3024++) {
            int key_3024 = arr_3024[i_3024];
            int j_3024 = i_3024 - 1;
            while (j_3024 >= 0 && arr_3024[j_3024] > key_3024) {
                arr_3024[j_3024 + 1] = arr_3024[j_3024];
                j_3024--;
            }
            arr_3024[j_3024 + 1] = key_3024;
        }
    }

    public static void main(String[] args) {
        int arr_3024[] = {23, 78, 45, 8, 32, 56, 1};
        int n_3024 = arr_3024.length;
        System.out.printf("array yang belum terurut:\n");
        for (int i_3024 = 0; i_3024 < n_3024; i_3024++)
```

```

        System.out.print(arr_3024[i_3024] + " ");
    System.out.println(" ");
    insertionsort_2511533024(arr_3024);
    System.out.printf("array yang terurut:\n");
    for (int i_3024 = 0; i_3024 < n_3024; i_3024++)
        System.out.print(arr_3024[i_3024] + " ");
    System.out.println(" ");
}
}

```

2.3 Hasil output.

array yang belum terurut:

23 78 45 8 32 56 1

array yang terurut:

1 8 23 32 45 56 78

2.4 Analisis hasil dan Teori :

Insertion Sort merupakan algoritma pengurutan data yang bekerja dengan cara mengambil satu elemen (*key*) lalu menyisipkannya ke posisi yang tepat pada bagian array yang sudah terurut, mirip seperti cara mengurutkan kartu di tangan. Pada program ini, array awal {23, 78, 45, 8, 32, 56, 1} ditampilkan terlebih dahulu dalam keadaan belum terurut. Kemudian fungsi `insertionsort_2511533024()` melakukan perulangan dari indeks ke-1 sampai akhir, membandingkan nilai *key* dengan elemen sebelumnya, lalu menggeser elemen yang lebih besar ke kanan hingga ditemukan posisi yang sesuai. Setelah proses selesai, program menampilkan hasil akhir array yang sudah terurut secara ascending yaitu 1 8 23 32 45 56 78. Algoritma ini cukup efektif untuk data kecil atau hampir terurut, namun memiliki kompleksitas waktu terburuk $O(n^2)$ jika data acak atau terbalik.

2.5 Langkah kerja program SelectionSort

- Buat di kelas operator dalam package pekan7.
- Deklarasikan class utama dengan nama SelectionSort_2511533024.
- Masukan kode program
- Run program.

2.6 Contoh kode program SelectionSort.

```
package pekan7_2511533024;

public class selectionsort_2511533024 {

    public static void selectionsort_2511533024(int[] arr_3024) {
        int n_3024 = arr_3024.length;
        for (int i_3024 = 0; i_3024 < n_3024; i_3024++) {
            int minindex_3024 = i_3024;
            for (int j_3024 = i_3024 + 1; j_3024 < n_3024; j_3024++) {
                if (arr_3024[j_3024] < arr_3024[minindex_3024]) {
                    minindex_3024 = j_3024;
                }
            }
            int temp_3024 = arr_3024[i_3024];
            arr_3024[i_3024] = arr_3024[minindex_3024];
            arr_3024[minindex_3024] = temp_3024;
        }
    }

    public static void main(String[] args) {
        int arr_3024[] = {23, 78, 45, 8, 32, 56, 1};
        int n_3024 = arr_3024.length;
        System.out.printf("array yang belum terurut:\n");
        for (int i_3024 = 0; i_3024 < n_3024; i_3024++)
            System.out.print(arr_3024[i_3024] + " ");
        System.out.println(" ");
    }
}
```



```

        selectionsort_2511533024(arr_3024);

        System.out.printf("array yang terurut:\n");
        for (int i_3024 = 0; i_3024 < n_3024; i_3024++)
            System.out.print(arr_3024[i_3024] + " ");
        System.out.println(" ");
    }
}

```

2.7 Hasil output :

array yang belum terurut:

23 78 45 8 32 56 1

array yang terurut:

1 8 23 32 45 56 78

2.8 Analisis hasil dan teori :

Selection Sort merupakan algoritma pengurutan yang bekerja dengan cara mencari nilai terkecil dari bagian array yang belum terurut, lalu menukarnya ke posisi paling depan secara bertahap. Pada program ini, array {23, 78, 45, 8, 32, 56, 1} ditampilkan terlebih dahulu dalam keadaan belum terurut. Kemudian fungsi `selectionsort_2511533024()` melakukan perulangan untuk menentukan indeks elemen terkecil (*minindex*) pada setiap tahap, lalu melakukan pertukaran (*swap*) antara elemen di posisi awal dengan elemen terkecil yang ditemukan. Proses ini terus dilakukan sampai seluruh elemen tersusun rapi. Hasil akhirnya array menjadi terurut secara ascending yaitu 1 8 23 32 45 56 78. Algoritma ini memiliki kompleksitas waktu $O(n^2)$ karena selalu melakukan pencarian minimum pada setiap iterasi, namun kelebihanannya adalah jumlah pertukaran data relatif sedikit dibanding algoritma sorting lainnya.

2.9 Langkah kerja program BubleSort

- Buat di kelas operator dalam package pekan7.
- Deklarasikan class utama dengan nama BubleSort_2511533024.
- Masukan kode program.
- Run program.

2.10 contoh kode program BubbleSort.

```
package pekan7_2511533024;

public class bubblesort_2511533024 {

    public static void bubblesort_2511533024(int[] arr_3024) {
        int n_3024 = arr_3024.length;
        for (int i_3024 = 0; i_3024 < n_3024; i_3024++) {
            for (int j_3024 = 0; j_3024 < n_3024 - i_3024 - 1; j_3024++) {
                if (arr_3024[j_3024] > arr_3024[j_3024 + 1]) {
                    int temp_3024 = arr_3024[j_3024];
                    arr_3024[j_3024] = arr_3024[j_3024 + 1];
                    arr_3024[j_3024 + 1] = temp_3024;
                    System.out.println("data : " + arr_3024[j_3024] + " " + arr_3024[j_3024 + 1]);
                }
            }
        }
    }

    public static void main(String[] args) {
        int arr_3024[] = {23, 78, 45, 8, 32, 56, 1};
        int n_3024 = arr_3024.length;
        System.out.print("array yang belum terurut: ");
        for (int i_3024 = 0; i_3024 < n_3024; i_3024++)
            System.out.print(arr_3024[i_3024] + " ");
        System.out.println(" ");
        bubblesort_2511533024(arr_3024);
        System.out.print("array yang terurut menggunakan bubblesort: ");
        for (int i_3024 = 0; i_3024 < n_3024; i_3024++)
            System.out.print(arr_3024[i_3024] + " ");
        System.out.println(" ");
    }
}
```

```
}  
}
```

2.12 Hasil output

array yang belum terurut: 23 78 45 8 32 56 1

data : 45 78

data : 8 78

data : 32 78

data : 56 78

data : 1 78

data : 8 45

data : 32 45

data : 1 56

data : 8 23

data : 1 45

data : 1 32

data : 1 23

data : 1 8

array yang terurut menggunakan bubblesort: 1 8 23 32 45 56 78

2.13 Analisis hasil dan teori

Bubble Sort merupakan algoritma pengurutan yang bekerja dengan cara membandingkan dua elemen yang bersebelahan, kemudian menukarnya jika urutannya salah. Proses ini dilakukan berulang-ulang sampai seluruh elemen berada pada posisi yang benar, sehingga nilai terbesar akan “mengambang” ke bagian akhir array pada setiap iterasi. Pada program ini, array awal {23, 78, 45, 8, 32, 56, 1} ditampilkan terlebih dahulu dalam keadaan belum terurut. Kemudian fungsi `bubblesort_2511533024()` menjalankan dua perulangan, dimana perulangan dalam membandingkan elemen `arr[j]` dan `arr[j+1]`, lalu melakukan pertukaran (*swap*) jika elemen sebelumnya lebih besar. Program juga menampilkan output "data : ..." setiap kali terjadi pertukaran untuk memperlihatkan proses pengurutan yang sedang berjalan. Setelah seluruh proses selesai, array berhasil diurutkan secara ascending menjadi 1 8 23 32 45 56 78. Bubble Sort memiliki kompleksitas waktu $O(n^2)$ sehingga kurang efisien untuk data besar, namun mudah dipahami dan cocok untuk pembelajaran dasar sorting.

2.14 Langkah kerja program InsertionSortGUI

- Buat di kelas operator dalam package pekan7.
- Deklarasikan class utama dengan nama InsertionSortGUI_2511533024.
- Masukan kode program.
- Run program.

2.15 Contoh kerja program InsertionSortGUI.

```
package pekan7_2511533024;

import java.awt.BorderLayout;
import java.awt.Color;
import java.awt.Dimension;
import java.awt.FlowLayout;
import java.awt.Font;
import javax.swing.BorderFactory;
import javax.swing.JButton;
import javax.swing.JFrame;
import javax.swing.JLabel;
import javax.swing.JOptionPane;
import javax.swing.JPanel;
import javax.swing.JScrollPane;
import javax.swing.JTextArea;
import javax.swing.JTextField;
import javax.swing.SwingConstants;
import javax.swing.SwingUtilities;

public class InsertionSortGUI_2511533024 extends JFrame {

    private static final long serialVersionUID = 1L;

    private int[] array_3024;

    private JLabel[] labelArray_3024;
```

```

private JButton stepButton_3024, resetButton_3024, setButton_3024;
private JTextField inputField_3024;
private JPanel panelArray_3024;
private JTextArea stepArea_3024;
private int i = 1, j;
private boolean sorting_3024 = false;
private int stepCount_3024 = 1;
public InsertionSortGUI_2511533024() {
    setTitle("Insertion Sort Langkah per Langkah");
    setSize(750, 400);
    setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
    setLocationRelativeTo(null);
    setLayout(new BorderLayout());

    // Panel input
    JPanel inputPanel = new JPanel(new FlowLayout());
    inputField_3024 = new JTextField(30);
    setButton_3024 = new JButton("Set Array");
    inputPanel.add(new JLabel("Masukkan angka (pisahkan dengan koma:"));
    inputPanel.add(inputField_3024);
    inputPanel.add(setButton_3024);

    // Panel array visual

    panelArray_3024 = new JPanel();
    panelArray_3024.setLayout(new FlowLayout());

    // Panel kontrol
    JPanel controlPanel = new JPanel();
    stepButton_3024 = new JButton("Langkah Selanjutnya");
    resetButton_3024 = new JButton("Reset");

```

```

stepButton_3024.setEnabled(false);
controlPanel.add(stepButton_3024);
controlPanel.add(resetButton_3024);

// Area teks untuk log langkah-langkah
stepArea_3024 = new JTextArea(8, 60);
stepArea_3024.setEditable(false);
stepArea_3024.setFont(new Font("Monospaced", Font.PLAIN, 14));
JScrollPane scrollPane = new JScrollPane(stepArea_3024);

// Tambahkan panel ke frame
add(inputPanel, BorderLayout.NORTH);
add(panelArray_3024, BorderLayout.CENTER);
add(controlPanel, BorderLayout.SOUTH);
add(scrollPane, BorderLayout.EAST);

// Event Set Array
setButton_3024.addActionListener(e -> setArrayFromInput_3024());
// Event Langkah Selanjutnya
stepButton_3024.addActionListener(e -> performStep_3024());
// Event Reset
resetButton_3024.addActionListener(e -> reset_3024());
}

private void setArrayFromInput_3024() {
    String text = inputField_3024.getText().trim();
    if (text.isEmpty())
        return;
    String[] parts = text.split(",");
    array_3024 = new int[parts.length];

```

```

try {
    for (int k = 0; k < parts.length; k++) {
        array_3024[k] = Integer.parseInt(parts[k].trim());
    }
} catch (NumberFormatException e) {
    JOptionPane.showMessageDialog(this, "Masukkan hanya angka yang dipisahkan
dengan koma!",
        "Error", JOptionPane.ERROR_MESSAGE);
    return;
}
i = 1;
stepCount_3024 = 1;
sorting_3024 = true;
stepButton_3024.setEnabled(true);
stepArea_3024.setText("");
panelArray_3024.removeAll();
labelArray_3024 = new JLabel[array_3024.length];
for (int k = 0; k < array_3024.length; k++) {
    labelArray_3024[k] = new JLabel(String.valueOf(array_3024[k]));
    labelArray_3024[k].setFont(new Font("Arial", Font.BOLD, 24));
    labelArray_3024[k].setBorder(BorderFactory.createLineBorder(Color.BLACK));
    labelArray_3024[k].setPreferredSize(new Dimension(50, 50));
    labelArray_3024[k].setHorizontalAlignment(SwingConstants.CENTER);
    panelArray_3024.add(labelArray_3024[k]);
}

panelArray_3024.revalidate();
panelArray_3024.repaint();
}
private void performStep_3024() {

```

```

if (i < array_3024.length && sorting_3024) {
    int key = array_3024[i];
    j = i - 1;
    StringBuilder stepLog = new StringBuilder();
    stepLog.append("Langkah ").append(stepCount_3024)
        .append(": Memasukkan ").append(key).append("\n");
    while (j >= 0 && array_3024[j] > key) {
        array_3024[j + 1] = array_3024[j];
        j--;
    }
    array_3024[j + 1] = key;
    updateLabels_3024();
    stepLog.append("Hasil: ").append(arrayToString_3024(array_3024)).append("\n\n");
    stepArea_3024.append(stepLog.toString());
    i++;
    stepCount_3024++;
    if (i == array_3024.length) {
        sorting_3024 = false;
        stepButton_3024.setEnabled(false);
        JOptionPane.showMessageDialog(this, "Sorting selesai!");
    }
}

private void updateLabels_3024() {
    for (int k = 0; k < array_3024.length; k++) {
        labelArray_3024[k].setText(String.valueOf(array_3024[k]));
    }
}

private void reset_3024() {

```



```

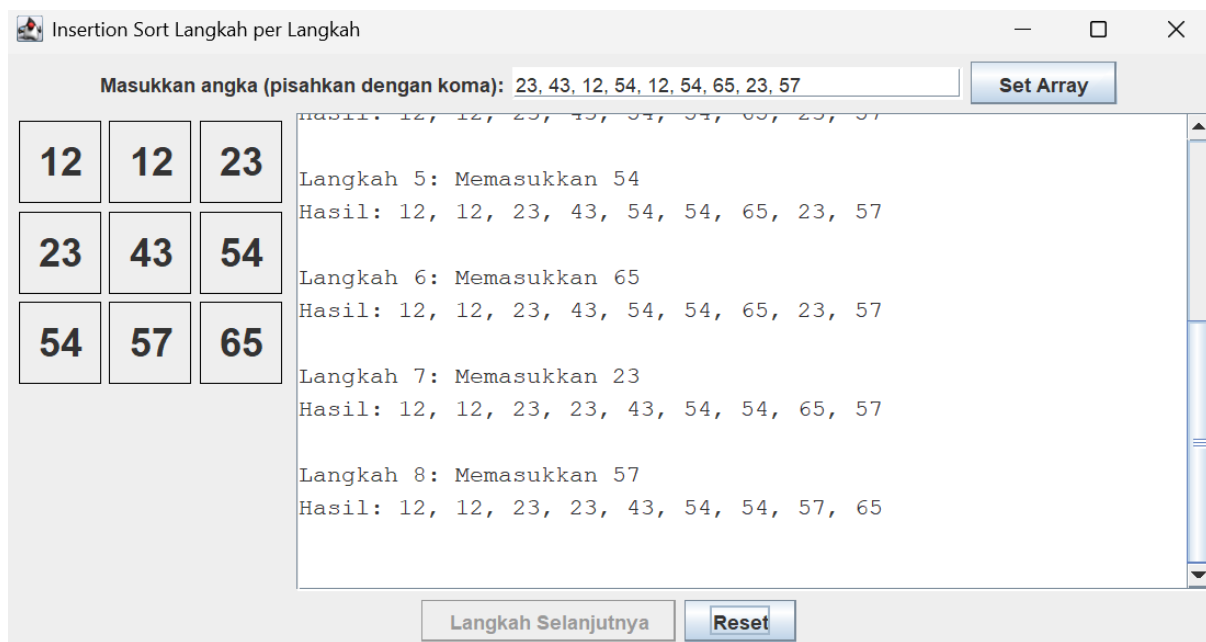
        inputField_3024.setText("");
        panelArray_3024.removeAll();
        panelArray_3024.revalidate();
        panelArray_3024.repaint();
        stepArea_3024.setText("");
        stepButton_3024.setEnabled(false);
        sorting_3024 = false;
        I = 1;
        stepCount_3024 = 1;
    }

    private String arrayToString_3024(int[] arr_3024) {
        StringBuilder sb = new StringBuilder();
        for (int k = 0; k < arr_3024.length; k++) {
            sb.append(arr_3024[k]);
            if (k < arr_3024.length - 1)
                sb.append(", ");
        }
        return sb.toString();
    }

    public static void main(String[] args) {
        SwingUtilities.invokeLater(() -> {
            InsertionSortGUI_2511533024 gui = new InsertionSortGUI_2511533024();
            gui.setVisible(true);
        });
    }
}

```

2.16 Hasil output



2.17 Analisis hasil dan teori.

Program InsertionSortGUI_2511533024 merupakan implementasi algoritma **Insertion Sort** berbasis **GUI (Graphical User Interface)** menggunakan Java Swing, yang bertujuan untuk menampilkan proses pengurutan data secara bertahap (*step by step*). Pada program ini, pengguna dapat memasukkan sekumpulan angka yang dipisahkan dengan koma melalui JTextField, lalu menekan tombol **Set Array** untuk menampilkan angka tersebut dalam bentuk label pada panel visual. Setelah array berhasil dimasukkan, tombol **Langkah Selanjutnya** akan aktif dan pengguna dapat menjalankan proses insertion sort satu langkah demi satu langkah. Setiap kali tombol ditekan, program mengambil elemen key dari indeks ke-*i*, kemudian membandingkannya dengan elemen sebelumnya dan menggeser elemen yang lebih besar ke kanan sampai ditemukan posisi yang tepat. Hasil pengurutan setiap langkah akan langsung diperbarui pada tampilan label dan juga dicatat pada JTextArea sebagai log proses, sehingga pengguna dapat memahami perubahan array pada tiap iterasi. Ketika seluruh data sudah terurut, program menonaktifkan tombol langkah dan menampilkan pesan bahwa sorting telah selesai. Selain itu, tombol **Reset** berfungsi untuk menghapus input, membersihkan tampilan array, serta mengulang kondisi program ke awal. Secara keseluruhan, program ini membantu memahami konsep Insertion Sort dengan lebih mudah karena menampilkan proses pengurutan secara visual dan terstruktur, meskipun algoritma insertion sort sendiri memiliki kompleksitas waktu $O(n^2)$ pada kasus terburuk.

BAB III

KESIMPULAN

Berdasarkan praktikum yang telah dilakukan mengenai algoritma sorting, dapat disimpulkan bahwa sorting merupakan proses penting dalam pengolahan data untuk menyusun elemen agar terurut secara ascending maupun descending. Algoritma Insertion Sort bekerja dengan cara menyisipkan elemen pada posisi yang tepat di bagian array yang sudah terurut, sehingga cocok digunakan untuk data yang jumlahnya kecil atau hampir terurut. Selection Sort melakukan pengurutan dengan mencari nilai terkecil dari data yang belum terurut kemudian menukarnya ke posisi depan secara bertahap, sehingga memiliki proses yang sederhana namun tetap memerlukan banyak perbandingan. Sedangkan Bubble Sort bekerja dengan cara membandingkan elemen yang bersebelahan dan menukarnya jika urutan salah, hingga seluruh data tersusun rapi. Ketiga algoritma ini memiliki kompleksitas waktu rata-rata $O(n^2)$, sehingga kurang efisien untuk data berukuran besar, namun sangat baik digunakan sebagai dasar pemahaman konsep sorting dalam struktur data.

DAFTAR PUSTAKA

- Modul pratikum pekan 7 – Sorting Algorithm.
- Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2009). *Introduction to Algorithms*. MIT Press.
- Munir, Rinaldi. (2012). *Algoritma dan Pemrograman*. Bandung: Informatika.
- Kadir, Abdul. (2014). *Dasar Pemrograman Java*. Yogyakarta: Andi Publisher.
- Sedgewick, Robert & Wayne, Kevin. (2011). *Algorithms*. Addison-Wesley.
- Pressman, Roger S. (2010). *Software Engineering: A Practitioner's Approach*. McGraw-Hill.